

Flask-SQLAlchemy Lab 2 (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p4-v2-flask-sqlalchemy-lab-2>  <https://github.com/learn-co-curriculum/python-p4-v2-flask-sqlalchemy-lab-2/issues/new>

Learning Goals

- Use Flask-SQLAlchemy to define a data model with relationships
 - Implement an association proxy for a model
 - Use SQLAlchemy-Serializer to serialize an object with relationships
-

Setup

Fork and clone the lab repo.

Run `pipenv install` and `pipenv shell` .

```
$ pipenv install
$ pipenv shell
```

Change into the `server` directory:

```
$ cd server
```

The file `server/models.py` defines models named `Customer` and `Item` .

Run the following commands to create the tables for customers and items.

```
$ flask db init
$ flask db migrate -m "initial migration"
$ flask db upgrade head
```

Task#1 : Add Review and relationships with Customer and Item



A customer can review an item that they purchased.

- A review **belongs to** an customer.
- A review **belongs to** an item.
- A customer **has many** items **through** reviews.
- An item **has many** customers **through** reviews.

Edit `server/models.py` to add a new model class named `Review` that inherits from `db.Model`. Add the following attributes to the `Review` model:

- a string named `__tablename__` assigned to the value `'reviews'`.
- a column named `id` to store an integer that is the primary key.
- a column named `comment` to store a string.
- a column named `customer_id` that is a foreign key to the `'customers'` table.
- a column named `item_id` that is a foreign key to the `'items'` table.
- a relationship named `customer` that establishes a relationship with the `Customer` model. Assign the `back_populates` parameter to match the property name defined to the reciprocal relationship in `Customer`.

- a relationship named `item` that establishes a relationship with the `Item` model. Assign the `back_populates` parameter to match the property name defined to the reciprocal relationship in `Item` .

Edit the `Customer` model to add the following:

- a relationship named `reviews` that establishes a relationship with the `Review` model. Assign the `back_populates` parameter to match the property name defined to the reciprocal relationship in `Review` .

Edit the `Item` model to add the following:

- a relationship named `reviews` that establishes a relationship with the `Review` model. Assign the `back_populates` parameter to match the property name defined to the reciprocal relationship in `Review` .

Save `server/models.py` . Make sure you are in the `server` directory, then type the following to perform a migration to add the new model:

```
$ flask db migrate -m 'add review'
$ flask db upgrade head
```

Test the new `Review` model class and relationships:

```
$ pytest testing/review_test.py
```

The 4 tests should pass. If not, update your `Review` model to pass the tests before proceeding.

Run `server/seed.py` to add sample customers, items, and reviews to the database.

```
$ python seed.py
```

Then use either Flask shell or SQLite Viewer to confirm the 3 tables are populated with the seed data.

Task #2: Add Association Proxy

Given a customer, we might want to get a list of items they've reviewed. Currently, you would need to iterate through the customer's reviews to get each item. Try this in the Flask shell:

```
>>> from models import *
>>> customer1 = Customer.query.filter_by(id=1).first()
>>> customer1
<Customer 1, Tal Yuri>
>>> items = [review.item for review in customer1.reviews]
>>> items
[<Item 1, Laptop Backpack, 49.99>, <Item 2, Insulated Coffee Mug, 9.99>]
>>>
```

Update `Customer` to add an association proxy named `items` to get a list of items through the customer's `reviews` relationship.

Once you've defined the association proxy, you can easily get the items for a customer as:

```
>>> customer1.items
[<Item 1, Laptop Backpack, 49.99>, <Item 2, Insulated Coffee Mug, 9.99>]
```

Test the updated `Customer` model and association proxy:

```
$ pytest testing/association_proxy_test.py
```

Task #3: Add Serialization

- Edit `Customer`, `Item`, and `Reviews` to inherit from `SerializerMixin`.
- Add serialization rules to avoid errors involving recursion depth (be careful about tuple commas).
 - `Customer` should exclude `reviews.customer`
 - `Item` should exclude `reviews.item`
 - `Review` should exclude `customer.reviews` and `item.reviews`

Test the serialized models:

```
$ pytest testing/serialization_test.py
```

Run all tests to ensure they pass:

```
$ pytest
```

Once all tests are passing, commit and push your work using `git` to submit.

This tool needs to be loaded in a new browser window

Load Flask-SQLAlchemy Lab 2 (CodeGrade) in a new window